1- CLASS TASKS SOLUTIONS

--- TASK 1 SOLUTION KEY ---

1. Logical Structure: if processing_hours <= 10.0: return "Low Risk" elif processing_hours <= 20.0: return "Medium Risk" else: return "High Risk"
2. Logic Error Explanation: If `hours > 10.0` is evaluated first, a value of `25.0` satisfies `> 10.0` and is incorrectly classified as "Medium Risk" instead of reaching the `> 20.0` "High Risk" check.

--- TASK 2 & 3 SOLUTION KEY (PYTHON CODE) ---

```Python
# Task 2: Function Definition
def classify_risk(processing_hours):
    try:
        hours = float(processing_hours)
    except (ValueError, TypeError):
        return "Invalid Data"

    if hours <= 10.0:
        return "Low Risk"
    elif hours <= 20.0:
        return "Medium Risk"
    else:
        return "High Risk"

# Task 2 Test Calls
print("Test 8.5:", classify_risk(8.5))          # Low Risk
print("Test 15.0:", classify_risk(15.0))        # Medium Risk
print("Test 25.0:", classify_risk(25.0))        # High Risk
print("Test MISSING:", classify_risk("MISSING")) # Invalid Data

# Task 3: Batch Loop Execution
dc_alpha_times = [12.0, 8.5, "CORRUPTED", 22.5, 19.0, 31.0, 9.0, 14.5,
"UNKNOWN", 27.0]

low_risk_count = 0
medium_risk_count = 0
high_risk_count = 0
invalid_count = 0

for item in dc_alpha_times:
    category = classify_risk(item)
    if category == "Low Risk":
        low_risk_count += 1
    elif category == "Medium Risk":
        medium_risk_count += 1
    elif category == "High Risk":
        high_risk_count += 1
    elif category == "Invalid Data":
        invalid_count += 1

total_valid_orders = low_risk_count + medium_risk_count + high_risk_count
```

```
print("\n--- TASK 3 BATCH RESULTS ---")
print("Low Risk Orders:   ", low_risk_count)    # 2 (8.5, 9.0)
print("Medium Risk Orders:", medium_risk_count) # 3 (12.0, 19.0, 14.5)
print("High Risk Orders:  ", high_risk_count)   # 3 (22.5, 31.0, 27.0)
print("Invalid Log Entries:", invalid_count)    # 2 ("CORRUPTED",
"UNKNOWN")
print("Total Valid Orders:", total_valid_orders)# 8

# Task 4: Metric & Memo
high_risk_pct = (high_risk_count / total_valid_orders) * 100
print(f"High Risk Percentage: {high_risk_pct:.2f}%") # 37.50%
```

--- TASK 4 SOLUTION KEY (BUSINESS MEMO) ---

"**To**: Marcus Vance, Operations Lead

Re: Distribution Center Alpha Risk Audit

An audit of Distribution Center Alpha's processing batch reveals that 37.5% of valid orders fall into the 'High Risk' category (>20 hours processing time), indicating substantial 2ulfilment delays. Furthermore, 20% of logged data entries were unparseable/corrupted. I recommend inspecting packing line capacity during peak hours and sanitizing timestamps at the facility level before configuring automated tracking."

2- HOMEWORK SOLUTION

```Python
# Homework Problem 1 & 2 Python Solution
ticket_batch = [
    {"ticket_id": "TCK-501", "type": "Urgent",  "resolution_hours": 2.5},
    {"ticket_id": "TCK-502", "type": "Routine", "resolution_hours": 18.0},
    {"ticket_id": "TCK-503", "type": "Urgent",  "resolution_hours": 6.0},
    {"ticket_id": "TCK-504", "type": "Routine", "resolution_hours":
"TIMEOUT_ERROR"},
    {"ticket_id": "TCK-505", "type": "Urgent",  "resolution_hours": 1.0},
    {"ticket_id": "TCK-506", "type": "Routine", "resolution_hours": 28.5},
    {"ticket_id": "TCK-507", "type": "Urgent",  "resolution_hours":
"NULL_VAL"},
    {"ticket_id": "TCK-508", "type": "Routine", "resolution_hours": 12.0},
    {"ticket_id": "TCK-509", "type": "Urgent",  "resolution_hours": 5.5},
    {"ticket_id": "TCK-510", "type": "Routine", "resolution_hours": 22.0}
]

def audit_ticket_sla(ticket_type, raw_hours):
    try:
        hours = float(raw_hours)
    except (ValueError, TypeError):
        return "Corrupted Data"
```

```
    if ticket_type not in ["Urgent", "Routine"]:
        return "Invalid Type"

    if ticket_type == "Urgent" and hours > 4.0:
        return "SLA Violation"
    elif ticket_type == "Routine" and hours > 24.0:
        return "SLA Violation"
    else:
        return "SLA Met"

# Execution Loop
total_eval = len(ticket_batch)
sla_met = 0
sla_violation = 0
corrupted = 0

urgent_met, urgent_viol = 0, 0
routine_met, routine_viol = 0, 0

print("--- TICKET SLA INDIVIDUAL LOG ---")
for t in ticket_batch:
    res = audit_ticket_sla(t["type"], t["resolution_hours"])
    print(f"Ticket {t['ticket_id']} ({t['type']}): {res}")

    if res == "SLA Met":
        sla_met += 1
        if t["type"] == "Urgent": urgent_met += 1
        else: routine_met += 1
    elif res == "SLA Violation":
        sla_violation += 1
        if t["type"] == "Urgent": urgent_viol += 1
        else: routine_viol += 1
    elif res == "Corrupted Data":
        corrupted += 1

valid_total = sla_met + sla_violation
success_rate = (sla_met / valid_total) * 100

print("\n--- SUMMARY METRICS ---")
print(f"Total Evaluated: {total_eval}")
print(f"Valid Tickets:   {valid_total}")
print(f"SLA Met:         {sla_met}")
print(f"SLA Violations:  {sla_violation}")
print(f"Corrupted Logs:  {corrupted}")
print(f"SLA Success Rate: {success_rate:.2f}%")
```

- **Homework Grading Criteria:**
    - Correctly identifies 8 valid tickets (2 corrupted: TCK-504, TCK-507).
    - Evaluates SLA Met = 5 (TCK-501, TCK-502, TCK-505, TCK-508, TCK-510) and SLA Violation = 3 (TCK-503: Urgent 6.0h; TCK-506: Routine 28.5h; TCK-509: Urgent 5.5h).
    - Calculates SLA Success Rate as 62.50% (5 / 8).
    - Identifies Urgent tickets as having a worse SLA breach rate (2 breaches out of 4 valid = 50% success) compared to Routine tickets (1 breach out of 4 valid = 75% success).